

```
//9.dhcp
//server
import java.net.*;
import java.util.*;
import java.io.*;
public class DHCPServer {
    private static final int SERVER_PORT = 4900;
    private static final String SERVER_IP = "127.0.0.1"; // Change to your server's IP
    private static final String IP_ALLOCATIONS_FILE = "ip_allocations.txt";
    private static List<String> availableIpAddresses = new ArrayList<>();
    private static Map<String, String> ipAllocations = new HashMap<>();
    public static void main(String[] args) {
        loadIpAllocations(); // Load IP allocations from file (if available)
        initializeIpAddresses();
        try {
            DatagramSocket socket = new DatagramSocket(SERVER_PORT);
            while (true) {
                byte[] receiveData = new byte[1024];
                DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
                socket.receive(receivePacket);
                // Simulate IP address allocation
                InetAddress clientAddress = receivePacket.getAddress();
                String macAddress = extractMacAddress(receiveData);
                String allocatedIp = allocateIpAddress(macAddress);
            }
        }
    }
}
```

```
String macAddress = extractMacAddress(receiveData);
String allocatedIp = allocateIpAddress(macAddress);
// Create and send DHCP response
byte[] responseData = createDHCPResponse(macAddress, allocatedIp);
DatagramPacket responsePacket = new DatagramPacket(responseData, responseData.length,
    clientAddress, receivePacket.getPort());
socket.send(responsePacket);
System.out.println("Allocated IP " + allocatedIp + " to client with MAC " + macAddress);
// Save IP allocations to file
saveIpAllocations();
}
} catch (Exception e) {
    e.printStackTrace();
}
}

private static void initializeIpAddresses() {
    // Simulate a pool of available IP addresses
    for (int i = 2; i <= 254; i++) {
        availableIpAddresses.add("192.168.1." + i);
    }
}

private static String extractMacAddress(byte[] data) {
    // Simulate extracting MAC address from DHCP request
```

```
File Edit Format View Help
private static String extractMacAddress(byte[] data) {
    // Simulate extracting MAC address from DHCP request
    // In a real implementation, you'd parse the DHCP request to get the MAC address
    return "18-66-DA-99-FE-05"; // Placeholder
}
```

```
private static String allocateIpAddress(String macAddress) {
    if (availableIpAddresses.isEmpty()) {
        return "No available IP addresses";
    }
}
```

```
    Random random = new Random();
    int index = random.nextInt(availableIpAddresses.size());
    String allocatedIp = availableIpAddresses.remove(index);
```

```
    // Bind MAC address with the allocated IP address
    ipAllocations.put(macAddress, allocatedIp);
```

```
    return allocatedIp;
}
```

```
private static byte[] createDHCPResponse(String macAddress, String allocatedIp) {
    // Simulate creating a DHCP response with the allocated IP address
    // In a real implementation, you'd construct a proper DHCP packet
}
```

```
// Simulate creating a DHCP response with the allocated IP address
// In a real implementation, you'd construct a proper DHCP packet
return ("Allocated IP: " + allocatedIp).getBytes();
}

private static void saveIpAllocations() {
    try (ObjectOutputStream outputStream = new ObjectOutputStream(new FileOutputStream(IP_ALLOCATIONS_FILE))) {
        outputStream.writeObject(ipAllocations);
        System.out.println("Saved IP allocations to " + IP_ALLOCATIONS_FILE);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void loadIpAllocations() {
    try (ObjectInputStream inputStream = new ObjectInputStream(new FileInputStream(IP_ALLOCATIONS_FILE))) {
        ipAllocations = (HashMap<String, String>) inputStream.readObject();
        System.out.println("Loaded IP allocations from " + IP_ALLOCATIONS_FILE);
    } catch (FileNotFoundException e) {
        System.out.println(IP_ALLOCATIONS_FILE + " not found. Starting with an empty IP allocations");
    } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
    }
}
```

ting a DHCP response with the allocated IP address
ementation, you'd construct a proper DHCP packet
ed IP: " + allocatedIp).getBytes();

```
saveIpAllocations() {  
    OutputStream outputStream = new ObjectOutputStream(new FileOutputStream(IP_ALLOCATIONS_FILE));  
    .writeObject(ipAllocations);  
    println("Saved IP allocations to " + IP_ALLOCATIONS_FILE);  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

```
loadIpAllocations() {  
    InputStream inputStream = new ObjectInputStream(new FileInputStream(IP_ALLOCATIONS_FILE));  
    Map<String, String> map = (HashMap<String, String>) inputStream.readObject();  
    println("Loaded IP allocations from " + IP_ALLOCATIONS_FILE);  
    } catch (FileNotFoundException e) {  
        println(IP_ALLOCATIONS_FILE + " not found. Starting with an empty IP allocations map.");  
    } catch (ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```